# Strategic Architecture and Autonomous Handover Blueprint for a Multi-Tenant Medical SaaS Platform in South Africa

## Executive Summary

The modernization of clinical practice management requires an architectural paradigm that balances the centralized maintainability required by software agencies with the decentralized ownership mandated by healthcare regulations and client autonomy. For a Next.js application developed using Domain-Driven Design (DDD) principles and deployed for multiple independent clinical practices within South Africa, the deployment strategy must simultaneously satisfy operational efficiency, strict regulatory compliance, and seamless business continuity. The core challenge presented is maintaining a single source of truth—a solitary GitHub repository—while deploying highly customized, fully isolated application instances for each clinical tenant, all while ensuring the client can unilaterally seize control of their infrastructure and billing without service interruption.

This report provides an exhaustive, expert-level blueprint for structuring, deploying, and transferring this Next.js application. It details the precise mechanisms required to provision independent environments across Vercel, Google Cloud Platform (GCP), Firebase, Daily.co, Resend, and Payfast. Crucially, the analysis focuses on the technical, cryptographic, and administrative protocols required to facilitate a zero-downtime handover of all services, empowering individual clinics to assume complete financial and operational ownership. Furthermore, the report rigorously aligns this deployment strategy with the Protection of Personal Information Act (POPIA) and the Health Professions Council of South Africa (HPCSA) guidelines regarding electronic medical record (EMR) sovereignty, retention, and security, utilizing localized cloud regions to mitigate cross-border data transfer liabilities.

## Domain-Driven Design Integration with Isolated Cloud Infrastructure

Standard Software-as-a-Service (SaaS) multi-tenancy typically involves multiple customers sharing a single database and application instance, differentiated logically via a tenantId foreign key. While this model simplifies database migrations and reduces overhead, it introduces catastrophic risks in the healthcare sector, including the potential for cross-tenant data leakage and highly complex compliance auditing. For the medical practice application under review, an infrastructure-level isolation strategy—often termed "Isolated Multi-Tenancy" or "Single-Codebase, Multi-Instance Architecture"—is overwhelmingly superior.

The application's core logic relies on strict Domain-Driven Design bounded contexts. In an

isolated deployment model, these contexts must map directly to independent infrastructure silos to ensure that a handover process does not require extracting a single clinic's data from a pooled database.

The **Clinical Records** bounded context, responsible for the patient's Electronic Medical Record (EMR) including the Condition, Allergy, and Medication aggregates, requires absolute cryptographic isolation. This is achieved by provisioning a dedicated Google Cloud Platform (GCP) project and corresponding Firebase/Firestore instance for every single clinic.[1] By establishing a unique Firestore database per tenant, the architecture ensures that a clinic's data is fundamentally inaccessible to any other clinic, utilizing dedicated GCP service accounts scoped strictly to that environment.

The **Revenue Cycle** context, encompassing the Invoice and Claim aggregates, dictates financial operations. To satisfy autonomous handover requirements, this context must interface exclusively with the specific clinic's Payfast Merchant ID.[2] Invoices generated within the Next.js application are stored in the isolated Firestore instance, and payment state mutations are driven by Payfast Instant Transaction Notification (ITN) webhooks directed to the clinic's specific Vercel deployment URL.[2]

The **Scheduling Access** context manages the Appointment and BookingSlot entities. Real-time scheduling synchronization relies on Firestore listener connections, while telehealth consultations trigger isolated Daily.co API calls. To maintain tenant isolation, the application dynamically generates Daily.co meeting tokens using clinic-specific API keys ingested as environment variables at runtime.[4]

The **Patient Identity** context serves as the CRM of the system. The Patient aggregate contains highly sensitive Special Personal Information (SPI), necessitating encryption at rest within the africa-south1 (Johannesburg) GCP region to strictly comply with POPIA's data sovereignty mandates.[5]

Finally, the **Tenant Management** context represents the SaaS multi-tenancy layer itself. Rather than existing as a traditional database aggregate, the Tenant model is abstracted into Vercel Environment Variables. These variables configure the clinic's unique slug, custom domain, and active subscription tier, driving the conditional rendering of Next.js feature flags without intermingling tenant configurations in a centralized database.[7]

# Edge Infrastructure and Frontend Deployment Strategy

Vercel serves as the edge network, hosting the Next.js frontend, Serverless Functions, and Edge Middleware. The deployment strategy must leverage Vercel's platform capabilities to link a single GitHub repository to multiple distinct deployment projects.

## Monorepo Linking and Concurrency Limits

Vercel allows multiple independent projects to be linked to the identical GitHub repository, ensuring that bug fixes and feature updates pushed to the main branch trigger parallel, isolated builds for all connected clinic deployments.[9] Every commit initiates a deployment pipeline that reads the specific project's environment variables, injecting the correct Firebase credentials, Payfast keys, and Daily.co tokens into the Next.js build.[10]

However, platform limits must be strictly monitored to maintain this architecture. The number of Vercel Projects that can be connected to a single Git repository is dictated by the agency's subscription tier. The Hobby Plan strictly limits repository linking to three (3) projects, rendering it unsuitable for an agency managing multiple clients.[9] The Vercel Pro Plan permits up to sixty (60) linked projects per Git repository, alongside increased concurrent build capacity.[11] Should the agency scale beyond sixty clinical tenants, an Enterprise plan is required to unlock custom repository linking thresholds.[11]

## Feature Flagging via Environment Variables

Because the same codebase serves all clients, the application must dynamically expose features based on the clinic's purchased subscription tier. Next.js supports robust environment variable management, embedding NEXT_PUBLIC_ variables into the static bundle during the build phase.[8]

By defining variables such as NEXT_PUBLIC_ENABLE_TELEHEALTH=true or NEXT_PUBLIC_MAX_BOOKING_SLOTS=50 within the Vercel project settings, the Next.js application can utilize tree-shaking during the compilation step to strip out unused code, optimizing performance for clinics on lower tiers.[13] Alternatively, for real-time feature toggling without requiring a full Vercel rebuild, the Next.js application can query a configuration document within the clinic's isolated Firestore database upon application initialization, utilizing React context providers or Next.js App Router middleware to conditionally route users or render specific UI components based on the retrieved Tenant aggregate status.[14]

## The Vercel Handover Protocol

The primary operational requirement is the ability for a clinic to unilaterally assume control of their application hosting and billing. Vercel provides native functionality to transfer projects between teams without incurring DNS downtime or workflow interruptions.[16]

The zero-downtime transfer requires meticulous orchestration. The client must first establish their own Vercel account (Hobby or Pro) and invite the agency's administrator to their newly formed Vercel Team.[16] The agency administrator, utilizing their ownership privileges, initiates the project transfer from the agency's workspace to the client's workspace.[16]

While custom domains and aliases transfer seamlessly, ensuring continuous uptime, stringent

security protocols prevent the migration of encrypted environment variables.[16] Consequently, the escrow handover process must include the secure transmission of all configuration variables to the client, who must manually re-enter them into their new Vercel Project Settings.[16] Furthermore, because the Vercel project relies on a centralized GitHub repository, the agency must transfer a fork or a standalone clone of the repository to a GitHub organization controlled by the client.[17] The client then authorizes their Vercel account to access this newly siloed repository, completing the decoupling process.[17]

# Data Persistence and Cloud Sovereignty

Firebase provides the backend-as-a-service (BaaS) layer, encompassing Firestore (NoSQL database), Firebase Authentication, and Cloud Storage for medical imaging and document retention. Every Firebase project is intrinsically linked to an underlying Google Cloud Platform (GCP) project, forming the foundation of the application's data sovereignty strategy.[19]

## Infrastructure Localization and the POPIA Mandate

The Protection of Personal Information Act (POPIA) regulates the processing of "Special Personal Information" (SPI), which explicitly includes data concerning health, biometrics, and sex life.[20] Section 72 of POPIA strictly prohibits the transfer of personal information outside of South African borders unless the receiving country has adequate data protection laws, or the data subject has provided explicit, informed consent, which is difficult to manage reliably at scale.[21]

To minimize legal exposure, simplify compliance auditing, and protect against the rising tide of healthcare-focused ransomware attacks—such as the catastrophic 2024 breach of the South African National Health Laboratory Service (NHLS)—all primary databases must reside within South African borders.[23] The agency must explicitly provision the GCP project, the Firestore database, and the Firebase Storage buckets within the africa-south1 (Johannesburg) region.[6] This localized deployment strategy not only satisfies data sovereignty requirements but also drastically reduces network latency for South African practitioners and patients interacting with the system.[5]

## GCP Identity and Access Management (IAM) Handover

Because the agency utilizes a "one GCP project per clinic" model, transferring billing and ownership is a highly standardized Identity and Access Management (IAM) procedure, completely avoiding the complexities of database extraction or data migration.[1]

To execute the financial and operational handover, the agency must navigate to the GCP IAM & Admin console for the specific clinic's project. The client's designated administrative email address is added to the project and assigned the highest privilege level: the Owner role

(roles/owner).[27]

Once the client accepts the invitation and authenticates into the GCP Console, they must establish their own Cloud Billing account.[27] The client navigates to the Billing management interface, selects the transferred project, and executes the "Change Billing" operation, linking the GCP and Firebase infrastructure directly to their corporate credit card or invoicing profile.[28] Upon confirming that the financial transition is successful, the client utilizes their Owner privileges to revoke the agency's IAM bindings, finalizing the cryptographic and financial handover and assuming absolute control over the africa-south1 data assets.[27]

# Artificial Intelligence and Vector Database Integration

The application integrates Gemini Large Language Models (LLMs) to power a vector database, likely utilized for semantic search across clinical histories, automated structuring of the Condition aggregate, or intelligent patient intake summarization.

## Vertex AI Configuration and Data Residency

Access to enterprise-grade Gemini models (such as Gemini 1.5 Flash and Gemini 1.5 Pro) is facilitated through Vertex AI within the Google Cloud ecosystem. By enabling the Vertex AI API within the exact same isolated GCP project that houses the clinic's Firebase instance, the agency ensures that LLM usage costs are natively isolated and tracked per client, eliminating the need to parse complex billing exports.[25]

Crucially, Vertex AI and the Gemini APIs are fully supported and provisioned within the africa-south1 (Johannesburg) region.[25] The Next.js application must be explicitly configured to route all AI inference requests to the regional endpoint (e.g., https://africa-south1-aiplatform.googleapis.com) rather than relying on global endpoints.[33] This strict regional routing ensures that prompts containing highly sensitive clinical context or patient identities submitted for vectorization do not transit through foreign servers in North America or Europe, thereby maintaining absolute compliance with POPIA's cross-border data transfer restrictions.[20]

## Model Handover Mechanics

Because Vertex AI is a managed service operating within the broader Google Cloud Platform, the handover of the LLM and vector database infrastructure happens concurrently with the Firebase and GCP handover. The billing metrics for computation-based input and output characters are automatically transferred when the client assumes ownership of the GCP Billing Account.[28] The client requires no separate accounts or API keys for the AI infrastructure, provided the Vercel environment variables continue to reference the localized GCP project ID.

# Telehealth and Real-Time Communications

The Scheduling Access bounded context integrates telehealth capabilities to facilitate online consultations between practitioners and patients. This requires ultra-low latency WebRTC streaming, robust hardware codec support, and strict compliance with global and local healthcare privacy laws.

## Daily.co Implementation Strategy

Daily.co provides specialized SDKs and REST APIs for embedding high-performance video calls directly into the Next.js frontend, alongside advanced features such as HIPAA-compliant cloud recording and an AI-powered Clinical Notes API.[35]

To align with the autonomous handover requirement, the agency must carefully structure the Daily.co integration. While Daily.co supports managing multiple sub-accounts or domains under a master agency structure [37], this creates an inextricable dependency on the agency's primary billing instrument. Instead, the optimal strategy is to mandate that each clinic establishes an independent, direct relationship with Daily.co from the project's inception.

The clinic registers for their own Daily.co account and executes a Business Associate Agreement (BAA) or a POPIA-compliant data processing agreement directly with the vendor, ensuring absolute legal protection regarding recorded medical sessions.[39] The clinic then generates an API key from their Daily.co developer dashboard and securely transmits it to the agency. The agency injects this specific key into the clinic's isolated Vercel environment variables.

## Access Control and API Handover

Within the Next.js application, the Appointment aggregate dictates the generation of telehealth sessions. The application utilizes the clinic's Daily.co API key to dynamically provision rooms via the REST API (POST https://api.daily.co/v1/rooms).[41] To secure the sessions, these rooms must be configured programmatically with the property "privacy": "private", ensuring that unauthorized users cannot guess the room URL and join the consultation.[4] The application subsequently utilizes the /meeting-tokens endpoint to generate secure, time-limited access tokens for the authenticated Patient and Practitioner, embedding these tokens into the frontend client.[4]

Because the clinic owns the Daily.co account and the billing instrument natively, the handover process is instantaneous and frictionless. If the agency ceases operations, the WebRTC infrastructure continues to function autonomously, drawing funds directly from the clinic's configured payment method without requiring any administrative transfers.

# Financial Infrastructure and the Revenue Cycle

The Revenue Cycle bounded context handles the financial lifecycle of the medical practice, specifically focusing on the Invoice aggregate. To process electronic funds transfers (EFT), credit card transactions, and medical aid co-payments in South African Rand (ZAR), the application integrates with Payfast, South Africa's premier payment gateway.[42]

### Gateway Integration and FICA Compliance

While advanced payment platforms offer complex multi-vendor split-payment architectures [43], treating the SaaS application as a master marketplace where the agency collects all patient funds and subsequently distributes them to clinics introduces insurmountable regulatory, tax, and liability burdens.

Instead, the agency must act purely as a technical integrator. South African financial regulations require all entities accepting payments to undergo rigorous verification under the Financial Intelligence Centre Act (FICA). Therefore, each clinic must independently register for a Payfast account, undergo FICA verification using their own corporate registration documents, and link their own business banking details for payouts.[2]

Once verified, the clinic navigates to their Payfast Developer Settings and retrieves their unique Merchant ID, Merchant Key, and a secure Passphrase.[2] The agency injects these three cryptographic identifiers into the clinic's isolated Vercel environment variables.

### Transaction Lifecycles and Autonomous Operations

When a Patient settles an Invoice, the Next.js application generates an HTML form or API request containing the specific clinic's Merchant ID and the transaction details, redirecting the user to the secure Payfast checkout portal.[3] Upon successful payment, Payfast triggers an Instant Transaction Notification (ITN)—a secure, server-to-server HTTP POST request—back to the Vercel application's API endpoint (e.g., https://clinic-domain.co.za/api/webhooks/payfast).[2] The application verifies the ITN signature using the clinic's Passphrase and updates the Invoice status to Paid within the isolated Firestore database.

Because the FICA registration, banking details, and transaction fee deductions (which Payfast extracts automatically at the point of sale) are entirely managed by the clinic, the financial infrastructure is entirely decoupled from the agency.[45] In a handover scenario, absolutely no migration of financial data or billing accounts is required. The application remains fully functional, and revenue continues to flow directly into the clinic's bank account.

## Communications and Bot Mitigation

### Transactional Email Delivery via Resend

Transactional emails, including appointment confirmations, telehealth links, and invoice

deliveries, are critical components of the patient experience. The application utilizes the Resend API to deliver these communications efficiently, leveraging modern REST principles and SDKs.[46]

Resend allows users to manage multiple distinct workspaces, or "Teams," under a single registered email address.[48] The agency leverages this architecture by creating a separate Team for each clinical tenant. Each Team receives its own isolated API keys, billing metrics, and usage quotas, preventing a high-volume clinic from exhausting the API limits of a smaller practice.[49] The agency configures the clinic's custom domain (e.g., mail.clinic-domain.co.za) by adding specific MX and TXT records to the Vercel DNS settings, ensuring emails are properly authenticated via SPF and DKIM to prevent spoofing and ensure high deliverability rates.[46]

**Handover Protocol:** Resend's Team architecture streamlines the client transfer process. The agency invites the clinic's administrative email address to their specific Resend Team, granting them the Admin role.[49] The clinic administrator then accesses the Resend Billing dashboard and inputs their corporate credit card, effectively replacing the agency's payment instrument.[51] Finally, the clinic utilizes their administrative privileges to revoke the agency's access to the Team, assuming total, exclusive control over the domain reputation, email logs, and API keys.[49]

### reCAPTCHA Integration and Security

To protect the Patient Identity context—specifically login portals, password resets, and public appointment booking forms—from automated bot abuse and credential stuffing attacks, the application implements Google reCAPTCHA.

Because reCAPTCHA Enterprise and reCAPTCHA v3 are managed services within the Google Cloud ecosystem, the site keys and secret keys are generated directly within the specific GCP project allocated to the clinic. Consequently, transferring the GCP project ownership (as detailed in the Data Persistence section) automatically transfers administrative control and billing responsibilities for the reCAPTCHA services to the client. The Vercel application continues to utilize the embedded keys without requiring any code modifications or downtime.

## South African Healthcare Compliance Framework

The deployment and management of a medical SaaS platform in South Africa is governed by a stringent regulatory framework designed to protect patient privacy and ensure the durability of clinical data. The architecture must be explicitly engineered to satisfy these mandates programmatically.

### Electronic Medical Record (EMR) Retention and Immutability

The Health Professions Council of South Africa (HPCSA) enforces rigorous guidelines regarding the lifecycle of patient health records, detailed comprehensively in Booklet 9: Guidelines on the Keeping of Patient Records.[52]

The application's Clinical Records bounded context must adhere to the following statutory retention periods:

1. **Standard Retention:** Adult patient records must be retained for a minimum of six (6) years from the date the record becomes dormant.[53]
2. **Minors:** Records concerning minors must be preserved continuously until the patient reaches their 21st birthday, regardless of dormancy.[54]
3. **Mental Impairment:** Records belonging to patients who are mentally impaired must be kept indefinitely, up to and including the patient's death.[54]
4. **Occupational Health:** Records pertaining to illnesses or accidents arising from a person's occupation carry a specialized retention mandate of twenty (20) years after treatment has concluded.[54]

Furthermore, the HPCSA mandates strict data immutability. Electronic systems must be designed to record information once, strictly preventing historical clinical entries from being overwritten or removed. If an alteration is required for patient care, a new note must be appended referring to the correction, while the original entry remains legible and auditable.[53]

**Architectural Implementation:**

To satisfy these requirements, the Next.js application must entirely abandon the use of hard deletes (e.g., standard HTTP DELETE requests) for the Condition, Allergy, Medication, and Patient aggregates. Instead, a robust soft-delete mechanism utilizing boolean flags (e.g., isArchived: true) and timestamping (archivedAt) must be embedded within the business logic.

More critically, this application-level logic must be backed by infrastructure-level enforcement. Firestore Security Rules must be authored to explicitly deny delete operations on all critical clinical collections, ensuring that even a compromised frontend client or a malicious authenticated user cannot permanently destroy medical data.[1] Updates to clinical records should ideally be implemented using an append-only event-sourcing model or by maintaining a dedicated ClinicalRevisions subcollection to track the complete chronological history of every diagnosis and prescription.

## POPIA Security Safeguards and Auditing

Under the "Security Safeguards" condition of POPIA, responsible parties must secure the integrity and confidentiality of personal information to prevent loss, damage, or unauthorized access.[57]

The isolated infrastructure paradigm intrinsically supports this mandate by ensuring that a

vulnerability or credential leak affecting one clinic does not compromise the data of another. Data is encrypted in transit by Vercel's automatic SSL/HTTPS provisioning [7] and encrypted at rest by Google Cloud's default storage protocols.[58]

To fulfill POPIA's accountability requirements, the system must maintain an immutable audit trail detailing which practitioner accessed or modified which patient record. This is achieved by implementing Next.js Edge Middleware or centralized backend services that intercept all GET, POST, and PATCH requests affecting the Clinical Records context, logging the user ID, timestamp, and action to a heavily restricted AuditLogs Firestore collection.

# Comprehensive Business Continuity and Handover Execution (The Runbook)

To satisfy the core requirement—enabling clients to manage their applications completely independently should the agency become incapacitated ("hit by a bus" scenario)—a comprehensive Business Continuity Plan (BCP) must be established at the very commencement of the client relationship.

The agency must prepare a secure **"Digital Escrow and Configuration Vault"** for each clinic. This vault, maintained in a secure password manager or an encrypted digital document possessed by the clinic's management, contains the architectural map and the cryptographic keys necessary to resurrect the application.

## Phase 1: Pre-Emptive Infrastructure Setup

The foundations of autonomous handover are laid before a single line of code is deployed:

1. **Domain Ownership:** The custom domain (e.g., dr-smith.co.za) must be registered directly in the client's name, utilizing the client's financial instrument, on a registrar like Xneelo or GoDaddy. The agency is granted temporary DNS zone modification rights to configure Vercel routing. If the agency disappears, the client retains absolute control over their internet identity.
2. **Service Independence:** The Payfast and Daily.co accounts are registered strictly under the clinic's corporate entity from day one, ensuring uninterrupted revenue flows and telehealth compliance regardless of the agency's operational status.

## Phase 2: The Emergency Action Plan

The escrow document must provide a precise, non-technical matrix for the clinic to execute a hostile takeover of the software lifecycle.

| Execution Phase | Required Action | Mechanism of Transfer |
|---|---|---|

| | | |
|---|---|---|
| **1. Codebase Recovery** | Secure the Next.js source code. | The agency provides a continuously updated mirror of the GitHub repository to a GitHub account controlled by the clinic, or grants the clinic download access to the private repository.[17] |
| **2. Backend Takeover** | Seize control of the databases and AI processing. | The clinic activates their pre-assigned GCP Owner IAM permissions.[27] They link their corporate credit card to the GCP Billing console, securing continuous access to Firestore, Vertex AI, and Cloud Storage.[31] |
| **3. Frontend Migration** | Migrate edge hosting to a new Vercel instance. | The clinic registers an independent Vercel account, links the recovered GitHub repository, and manually inputs the environment variables provided in the Escrow Document.[16] They then update their domain registrar to point to the new Vercel nameservers. |
| **4. Communications Control** | Secure the transactional email API. | The clinic utilizes their pre-assigned Admin privileges in the Resend Team dashboard to remove the agency's users and update the billing instrument, preventing service interruption.[49] |

# Conclusion

The deployment of a unified Next.js codebase to fully autonomous, client-owned

infrastructure is a highly sophisticated architectural strategy that represents the gold standard for medical SaaS platforms. By rejecting the traditional shared-database multi-tenant model in favor of an isolated multi-deployment paradigm, the agency achieves unparalleled data security, simplifies compliance with South Africa's stringent POPIA and HPCSA regulations, and permanently eliminates single points of failure.

Vercel's Git-linking capabilities form the robust backbone of the deployment pipeline, allowing a single, highly maintainable GitHub repository to power dozens of bespoke clinical environments through the intelligent application of environment variables. Concurrently, GCP's granular Identity and Access Management framework and Resend's multi-team architecture provide clear, highly auditable pathways for transferring financial and operational ownership directly to the client. By deploying Firebase and Vertex AI resources exclusively within the africa-south1 region, the architecture guarantees absolute data sovereignty and mitigates the severe liabilities associated with cross-border health data transfers.

Ultimately, this comprehensive architectural blueprint enables the agency to operate with the agility and scale of a centralized software provider, while simultaneously offering their medical clients the uncompromised security, operational autonomy, and long-term business continuity of a bespoke, self-hosted enterprise application.

**Works cited**


1. General best practices for setting up Firebase projects - Google, accessed February 15, 2026, https://firebase.google.com/docs/projects/dev-workflows/general-best-practices
2. Payfast Payment Gateway Documentation - WooCommerce, accessed February 15, 2026, https://woocommerce.com/document/payfast-payment-gateway/
3. PayFast Developer Documentation>, accessed February 15, 2026, https://developers.payfast.co.za/
4. An introduction to room access control with Daily API meeting tokens, accessed February 15, 2026, https://www.daily.co/blog/intro-to-room-access-control/
5. Global Locations - Regions & Zones | Google Cloud, accessed February 15, 2026, https://cloud.google.com/about/locations
6. Cloud Firestore locations | Firebase - Google, accessed February 15, 2026, https://firebase.google.com/docs/firestore/locations
7. Vercel for Platforms, accessed February 15, 2026, https://vercel.com/docs/multi-tenant
8. Guides: Environment Variables - Next.js, accessed February 15, 2026, https://nextjs.org/docs/pages/guides/environment-variables
9. Using Monorepos - Vercel, accessed February 15, 2026, https://vercel.com/docs/monorepos
10. One Repository With Multiple Deployments, Environment Variables, and Secrets on Vercel?, accessed February 15, 2026, https://stackoverflow.com/questions/61978288/one-repository-with-multiple-deployments-environment-variables-and-secrets-on

11. github - Vercel maximum number of projects? - Stack Overflow, accessed February 15, 2026, https://stackoverflow.com/questions/70924026/vercel-maximum-number-of-projects
12. Limits - Vercel, accessed February 15, 2026, https://vercel.com/docs/limits
13. Feature flags in Next.js: How to manage feature rollouts efficiently - Statsig, accessed February 15, 2026, https://www.statsig.com/perspectives/feature-flags-nextjs-rollouts
14. Next.js Environment Switching in 2025: Build Once, Deploy Anywhere | by GDSKS - Medium, accessed February 15, 2026, https://gdsks.medium.com/next-js-environment-switching-in-2025-build-once-deploy-anywhere-efe76c55c09f
15. Multi-Tenant Architecture in Next.js: A Complete Guide | by Its Aman Yadav | Medium, accessed February 15, 2026, https://medium.com/@itsamanyadav/multi-tenant-architecture-in-next-js-a-complete-guide-25590c052de0
16. Transferring a project - Vercel, accessed February 15, 2026, https://vercel.com/docs/projects/transferring-projects
17. How do you hand off a project to a client? · vercel vercel · Discussion #5057 - GitHub, accessed February 15, 2026, https://github.com/vercel/vercel/discussions/5057
18. Best practices for organizing work in your enterprise - GitHub Docs, accessed February 15, 2026, https://docs.github.com/enterprise-cloud@latest/admin/overview/best-practices-for-enterprises
19. Understand Firebase projects - Google, accessed February 15, 2026, https://firebase.google.com/docs/projects/learn-more
20. POPIA Compliance for Health Data in South Africa, accessed February 15, 2026, https://www.webberwentzel.com/News/Pages/popia-compliance-for-health-data-navigating-special-personal-information-requirements-in-healthcare.aspx
21. South Africa Data Privacy - Amazon Web Services (AWS), accessed February 15, 2026, https://aws.amazon.com/compliance/south-africa-data-privacy/
22. Cross-border data transfers under the protection of personal information Act 4 of 2013, accessed February 15, 2026, https://www.dentons.com/en/insights/articles/2022/october/13/cross-border-data-transfers-under-the-protection-of-personal-information-act-4-of-2013
23. Data privacy and protection in AI-driven healthcare - SciELO South Africa, accessed February 15, 2026, https://www.scielo.org.za/pdf/samj/v115n5b/06.pdf
24. Data privacy and protection in AI-driven healthcare - SciELO South Africa, accessed February 15, 2026, https://scielo.org.za/pdf/samj/v115n5b/06.pdf
25. Vertex AI pricing | Google Cloud, accessed February 15, 2026, https://cloud.google.com/vertex-ai/pricing
26. Regions and zones | Filestore - Google Cloud Documentation, accessed February 15, 2026, https://docs.cloud.google.com/filestore/docs/regions
27. How to transfer Google Cloud project ownership? - Stack Overflow, accessed

February 15, 2026, https://stackoverflow.com/questions/25697766/how-to-transfer-google-cloud-project-ownership
28. Changing the Ownership of a Google Cloud Project: A Complete Guide | by Denis Bélanger, accessed February 15, 2026, https://medium.com/@python-javascript-php-html-css/changing-the-ownership-of-a-google-cloud-project-a-complete-guide-dd07e36c2ef5
29. Transferring project to another account - firebase - Stack Overflow, accessed February 15, 2026, https://stackoverflow.com/questions/37354880/transferring-project-to-another-account
30. Account Migration FAQ - Google Cloud Platform Console Help, accessed February 15, 2026, https://support.google.com/cloud/answer/7523192?hl=en
31. Enable, disable, or change billing for a project - Google Cloud Documentation, accessed February 15, 2026, https://docs.cloud.google.com/billing/docs/how-to/modify-project
32. How to Transfer a Google Cloud Project Between Accounts with Billing Ownership?, accessed February 15, 2026, https://discuss.google.dev/t/how-to-transfer-a-google-cloud-project-between-accounts-with-billing-ownership/177133
33. Deployments and endpoints | Generative AI on Vertex AI | Google Cloud Documentation, accessed February 15, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/locations
34. Vertex AI Pricing | Google Cloud, accessed February 15, 2026, https://cloud.google.com/vertex-ai/generative-ai/pricing
35. Supporting care with voice, video and AI - Daily, accessed February 15, 2026, https://www.daily.co/healthcare/
36. Telehealth - Realtime voice, video, and AI for developers - Daily, accessed February 15, 2026, https://www.daily.co/features/telehealth/
37. Get started with Daily - Daily | API documentation, accessed February 15, 2026, https://docs.daily.co/get-started
38. Partners - Daily.co, accessed February 15, 2026, https://www.daily.co/partners/
39. Daily HIPAA-compliant video chat, accessed February 15, 2026, https://docs.daily.co/guides/privacy-and-security/hipaa
40. HIPAA compliance details, for the Daily video call API, accessed February 15, 2026, https://www.daily.co/blog/hipaa-compliance-details-for-the-daily-co-video-call-api/
41. Video call API tutorial: the /rooms family of endpoints - Daily.co, accessed February 15, 2026, https://www.daily.co/blog/video-call-api-tutorial-the-rooms-family-of-endpoints/
42. Payfast - Free Invoicing Software for Small Businesses, accessed February 15, 2026, https://invoiceninja.com/payment-gateway/payfast/
43. Build a Marketplace Payment Solution with PayFast Split Payments - CMARIX, accessed February 15, 2026,

https://www.cmarix.com/blog/build-marketplace-payment-solution-with-payfast-split-payments/
44. Online order forms with PayFast integration - 123FormBuilder, accessed February 15, 2026, https://www.123formbuilder.com/docs/payfast/
45. Setting Up PayFast (South Africa) As Your Payment Gateway, accessed February 15, 2026, https://bootstrap.brilliantdirectories.com/support/solutions/articles/12000043027-setting-up-payfast-south-africa-as-your-payment-gateway
46. Resend · Email for developers, accessed February 15, 2026, https://resend.com/
47. Introduction - Resend, accessed February 15, 2026, https://resend.com/docs/api-reference/introduction
48. Managing Teams - Resend, accessed February 15, 2026, https://resend.com/docs/dashboard/settings/team
49. Multiple Teams - Resend, accessed February 15, 2026, https://resend.com/blog/multiple-teams
50. Vercel - Resend, accessed February 15, 2026, https://resend.com/docs/knowledge-base/vercel
51. Managing Billing - Resend, accessed February 15, 2026, https://resend.com/docs/dashboard/settings/billing
52. Guidelines on patient recordkeeping - - - Protecting the public and guiding the professions, accessed February 15, 2026, https://www.hpcsa-blogs.co.za/guidelines-on-patient-recordkeeping/
53. Patient Records - HPCSA ethical principles and professional conduct -, accessed February 15, 2026, https://www.hpcsa-blogs.co.za/patient-records-hpcsa-ethical-principles-and-professional-conduct/
54. Retention of medical records - Factsheets, accessed February 15, 2026, https://www.medicalprotection.org/southafrica/casebook-and-resources/factsheets/factsheets/sa-retention-of-medical-records
55. HIPAA Retention Requirements - 2026 Update, accessed February 15, 2026, https://www.hipaajournal.com/hipaa-retention-requirements/
56. Guideline on Medical Records in Industry - THE SOUTH AFRICAN SOCIETY OF OCCUPATIONAL MEDICINE, accessed February 15, 2026, https://sasom.org/wp-content/uploads/2023/08/16-Guideline-on-Medical-Records-in-Industry.pdf
57. Oracle Cloud comes to Johannesburg: Meeting compliance obligations in South Africa, accessed February 15, 2026, https://blogs.oracle.com/cloud-infrastructure/oracle-cloud-comes-to-johannesburg-meeting-compliance-obligations-in-south-africa
58. POPIA Compliance in South Africa: Everything Software Developers Should Know, accessed February 15, 2026, https://www.zealousys.com/blog/popia-compliance-in-south-africa-a-guide-for-software-developers/